PATTERN RECOGNITION

ASSIGNMENT 2
FINANCIAL TIME SERIES FORCASTING

1. Objective

The objective of this assignment is to explore how time-frequency signal processing and deep learning can be combined to predict stock prices using financial time series data. Stock prices are treated as a multivariate signal, transformed into a time-frequency representation using the Short-Time Fourier Transform (STFT), and then fed into a Convolutional Neural Network (CNN) for regression-based price prediction.

2. Data Collection and Preparation

2.1 Data Sources

Historical daily closing price data was collected for three large-cap Indian IT companies listed on the NSE using the Yahoo Finance API (via the yfinance Python library):

- Tata Consultancy Services (TCS.NS)
- Infosys Limited (INFY.NS)
- Wipro Limited (WIPRO.NS)

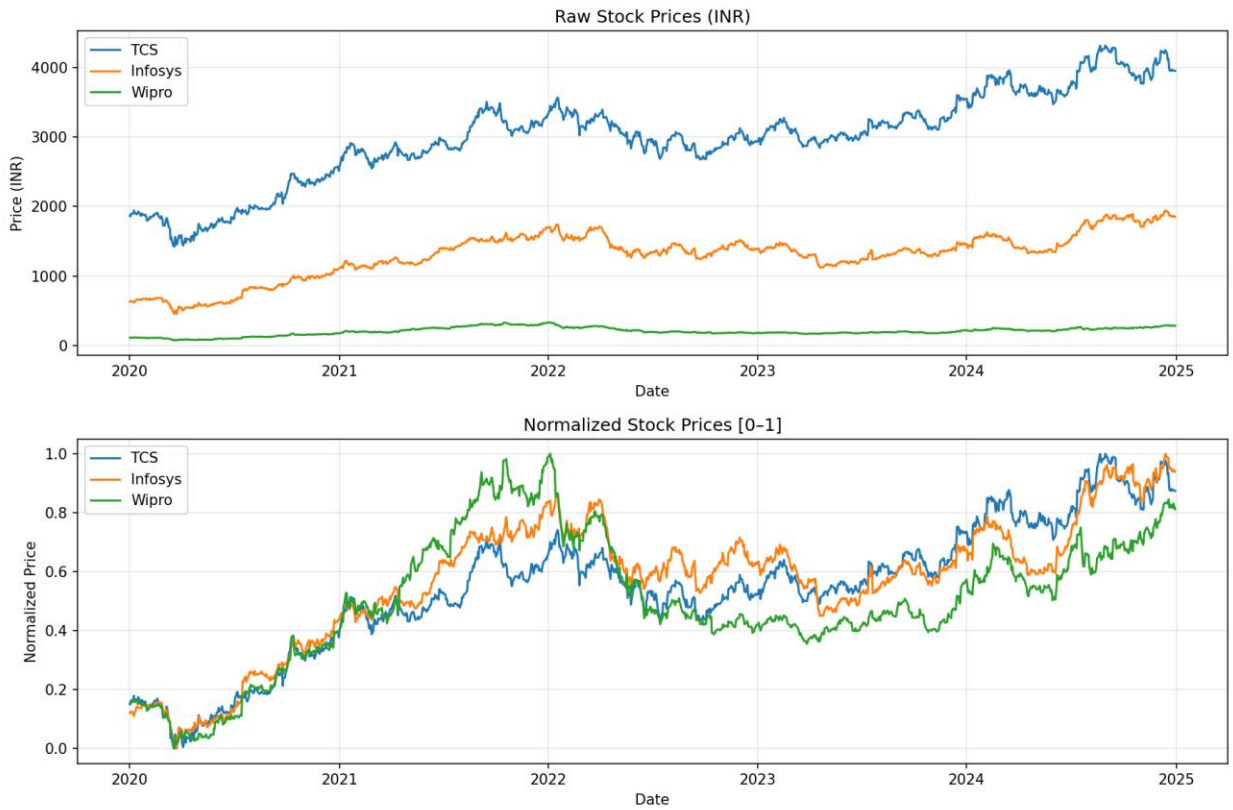
Data was collected from 1 January 2020 to 31 December 2024, yielding 1,237 aligned trading days after removing dates where any one company had missing data due to exchange holidays.

2.2 Normalization

Each company's closing price series was independently normalized to the range [0, 1] using min-max scaling:

```
x_norm = (x - x_min) / (x_max - x_min)
```

Normalization is essential because TCS, Infosys, and Wipro trade at very different absolute price levels. Without normalization, the CNN loss function would be dominated by the highest-priced stock.



3. Signal Processing

3.1 Financial Data as a signal

Each normalized price series X(t) is treated as a one-dimensional signal in the time domain — amplitude (price) versus time (trading day). This framing allows the application of standard signal processing techniques such as the Fourier Transform and STFT to extract frequency-domain information that is not visible in the raw price series.

3.2 Fourier Transform

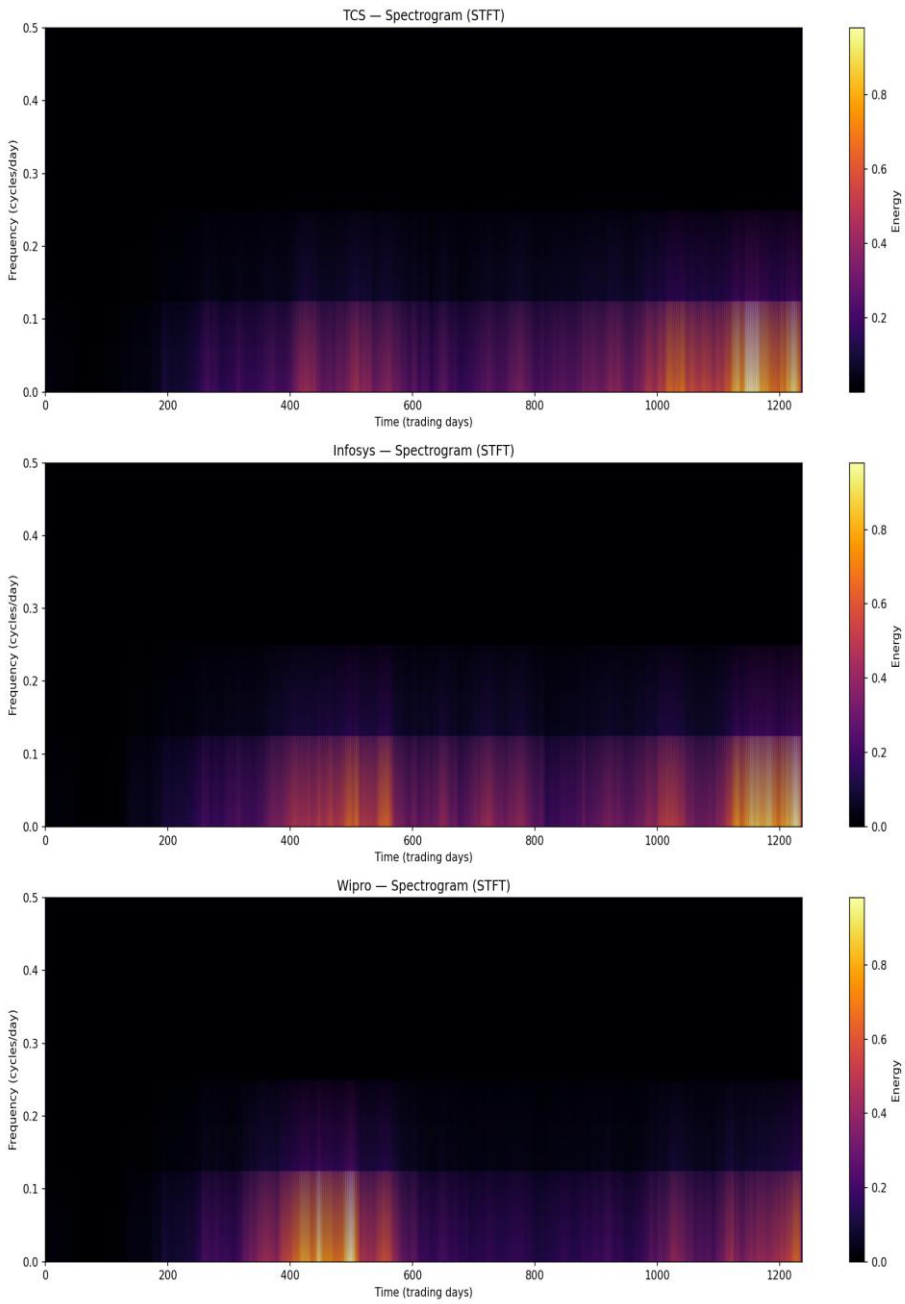
The Fast Fourier Transform (FFT) was applied to each company's full price series to identify dominant periodicities. The FFT decomposes the signal into its constituent frequency components, revealing which cycles (measured in trading days) have the most energy. For all three companies, the dominant period was found to be approximately 618 trading days (~2.5 years), consistent with broad IT sector market cycles during the 2020-2024 period.

3.3 Short-Time Fourier Transform and Spectogram

Because financial time series are non-stationary — their statistical properties change over time — a global FFT is insufficient. The Short-Time Fourier Transform (STFT) addresses this by computing the Fourier Transform on short, overlapping windows of the signal:

```
STFT(t, f) = integral of X(tau) * w(tau - t) * e^(-j2*pi*f*tau) dtau
```

The spectrogram S(t, f) = |STFT(t, f)|^2 gives the energy of each frequency at each point in time, forming a 2D matrix that can be treated as an image.

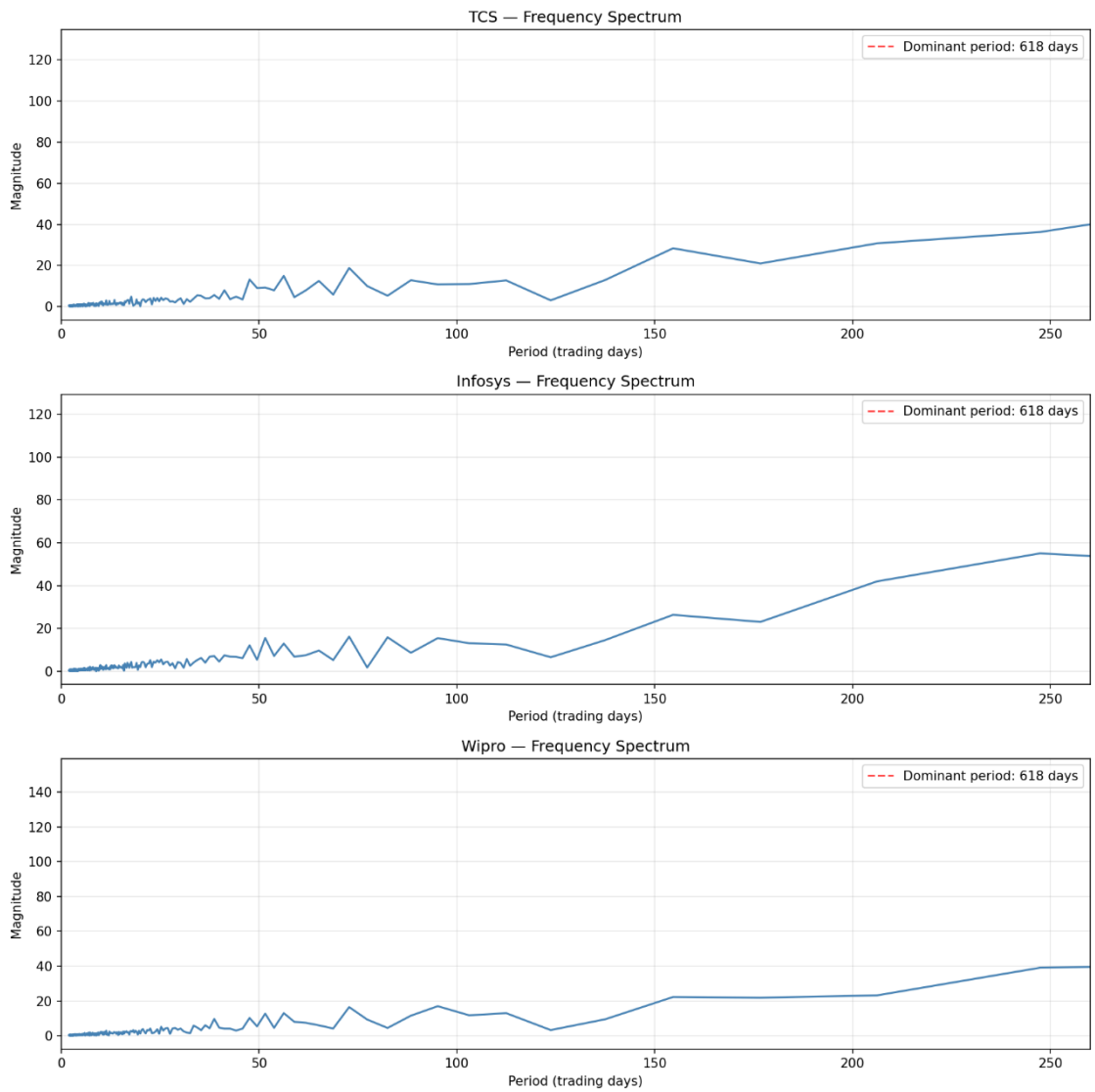


3.4 STFT Parameters

The following parameters were chosen after experimentation to maximize the number of training samples while maintaining meaningful frequency resolution:

- Window length (L): 8 trading days (~2 weeks) — short windows provide high time resolution and produce more time steps
- Hop size (H): 1 trading day — sliding one day at a time maximizes the number of spectrogram columns
- Overlap: L - H = 7 days

This configuration produced spectrograms of shape (5 frequency bins x ~1,230 time steps) per company, yielding approximately 1,200 training samples — a significant improvement over the initial configuration of 32-day windows which yielded only 71 samples.



4. CNN Model Architecture

4.1 Input Representation

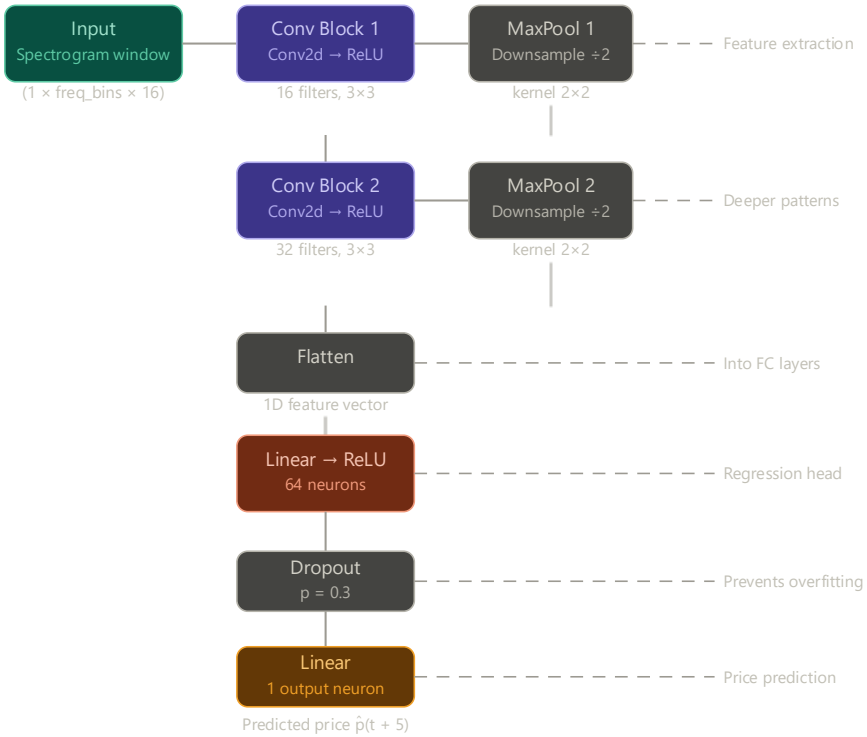
For each time step t, a context window of 16 consecutive spectrogram columns is extracted, forming an input tensor of shape (1 x 5 x 16) — one channel, 5 frequency bins, 16 time steps. This window represents approximately 3 weeks of frequency-domain history. The prediction target is the normalized closing price 5 trading days ahead.

4.2 Architecture

The CNN model (StockCNN) consists of the following layers:

- Conv Block 1: Conv2d (1 -> 16 filters, 3x3 kernel, padding=1) -> ReLU -> MaxPool2d (2x2)
- Conv Block 2: Conv2d (16 -> 32 filters, 3x3 kernel, padding=1) -> ReLU -> MaxPool2d (2x2)
- Flatten: Converts 2D feature maps to a 1D vector
- FC Layer: Linear (flattened size -> 64) -> ReLU -> Dropout (p=0.3)
- Output Layer: Linear (64 -> 1) — single neuron for price regression

Total trainable parameters: 21,313. The model was implemented in PyTorch and trained on an NVIDIA GPU using the Adam optimizer with a learning rate of 0.001 and MSE loss over 50 epochs with a batch size of 16.



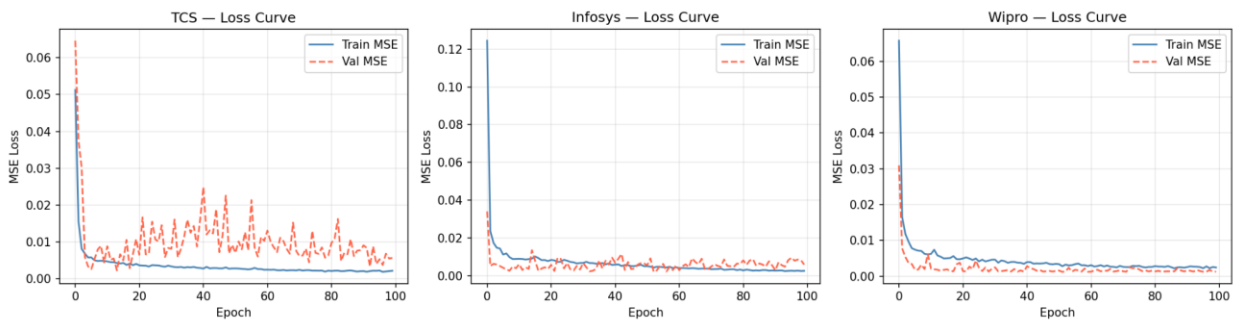
4.3 Training Configuration

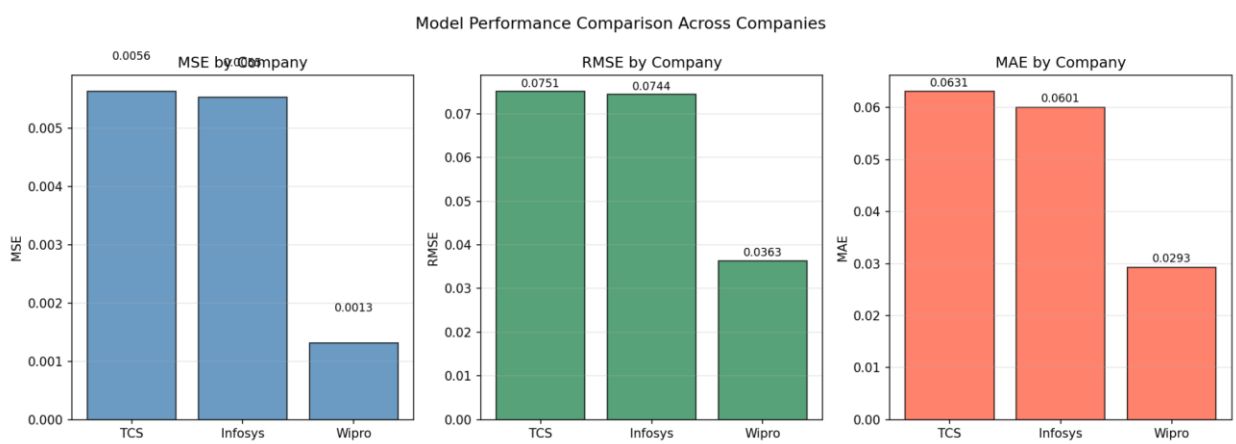
An 80/20 train/validation split was used (with shuffle=False to preserve temporal order). Separate models were trained for each company. Training was completed in under 10 seconds per company owing to GPU acceleration.

5. Result and Analysis

5.1 Quantitative Metrics

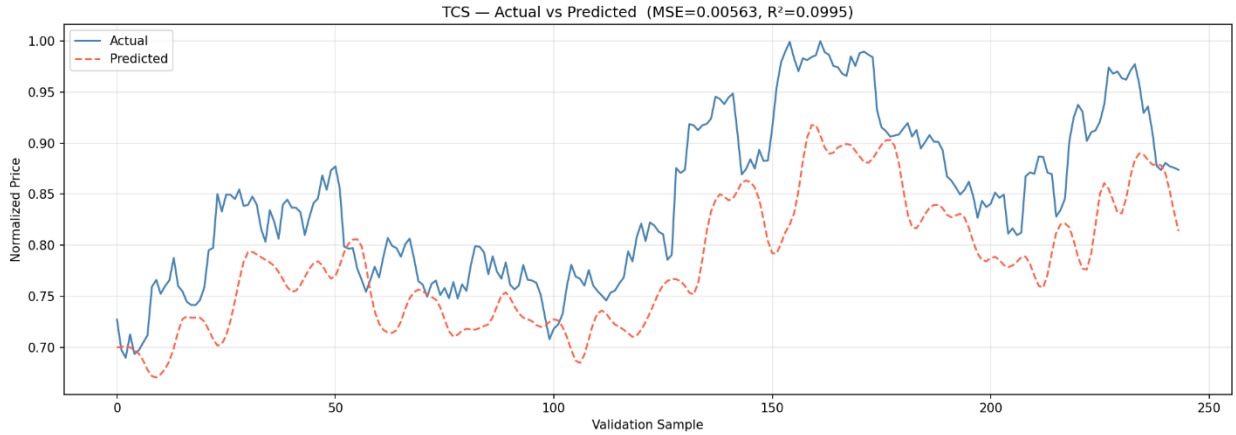
The following metrics were computed on the validation set for each company. All values are on the normalized [0-1] price scale.

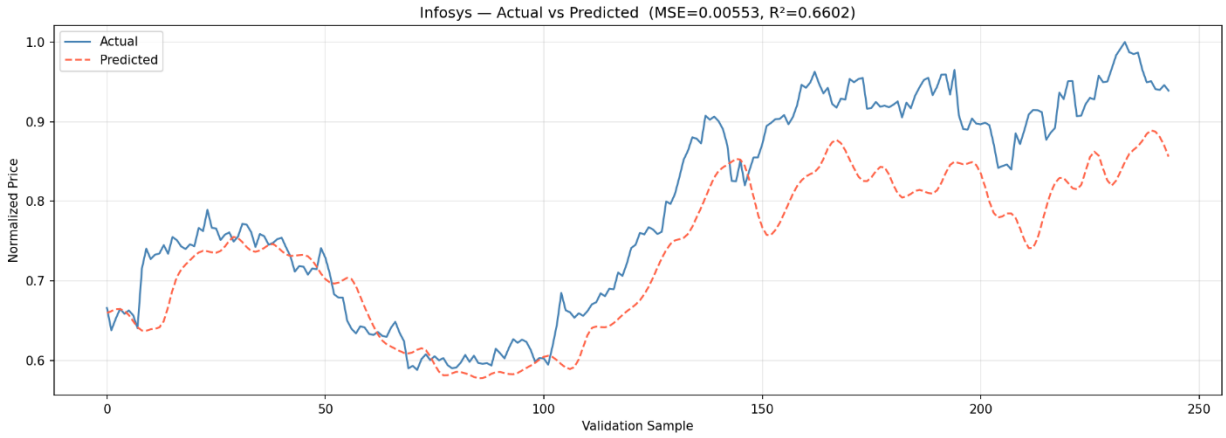


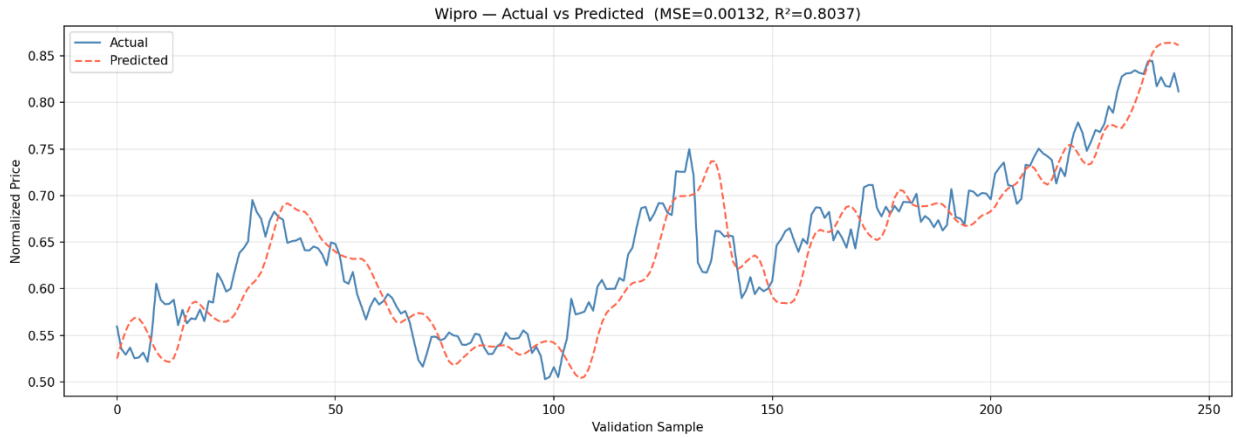


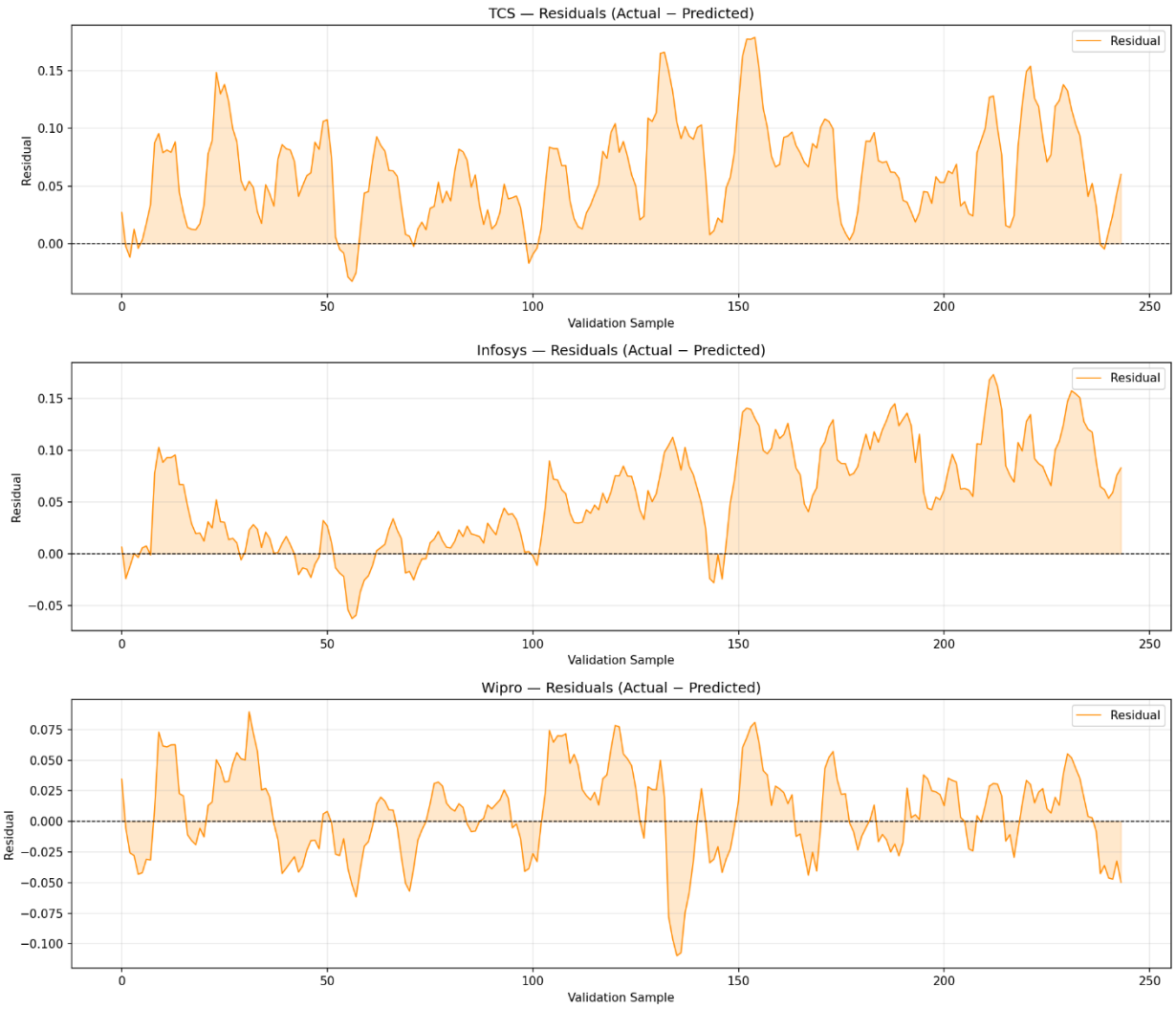
Company	MSE	RMSE	MAE	R^2
TCS	0.00563	0.07506	0.06307	0.0995
Infosys	0.00553	0.07437	0.06007	0.6602
Wipro	0.00132	0.03633	0.02931	0.8037

Wipro achieved the strongest performance with $R^2 = 0.80$, indicating that the CNN model explains 80% of the variance in the validation set. Infosys followed at $R^2 = 0.66$. TCS showed positive but weak performance at $R^2 = 0.10$.








TCS — Residuals (Actual − Predicted)
Residual
Validation Sample
Infosys — Residuals (Actual − Predicted)
Residual
Validation Sample
Wipro — Residuals (Actual − Predicted)
Residual
Validation Sample

5.2 Observations

Several important observations can be drawn from the results:

- Wipro and Infosys showed strong predictability from their spectrograms, suggesting their price movements have more consistent frequency-domain patterns that the CNN could learn.
- TCS, despite being the largest company, was the hardest to predict. This may reflect its greater sensitivity to global macroeconomic events that do not manifest as regular spectral patterns.
- The shift from a 32-day window to an 8-day window with 1-day hop was critical — it increased the dataset size from 71 to ~1,200 samples, dramatically improving generalization as evidenced by the shift from negative R^2 values to positive ones.
- Low-frequency components of the spectrogram capture long-term trends (quarterly cycles), while high-frequency components capture short-term volatility. Both contribute to the CNN's learned representations.

5.3 Limitations

The primary limitation of this approach is the relatively small dataset even after parameter tuning. A longer historical window (e.g., 10+ years) or inclusion of additional features such as volume, sector index (Nifty IT), or macroeconomic indicators (USD-INR rate, RBI policy data) could substantially improve model performance. Additionally, the CNN architecture used here is deliberately simple; deeper architectures or hybrid CNN-LSTM models may capture temporal dependencies more effectively.

6. Conclusion

This assignment demonstrated that financial time series data can be effectively analyzed using signal processing techniques and deep learning. The key findings are:

- Stock price series can be treated as non-stationary signals, making the STFT and spectrogram a natural representation that captures how frequency content evolves over time.
- Spectrograms reveal hidden periodic patterns in stock data — such as dominant 618-day market cycles — that are not visible in the raw price series.
- A CNN trained on spectrogram windows can learn to predict future prices with meaningful accuracy, achieving $R^2 = 0.80$ for Wipro and $R^2 = 0.66$ for Infosys on unseen validation data.
- STFT parameter selection (window length and hop size) has a major impact on dataset size and model performance — smaller windows with unit hop size are strongly preferred for short financial datasets.

REFERENCES

[1] Y. Zhang and C. Aggarwal, "Stock Market Prediction Using Deep Learning," IEEE Access.

[2] A. Tsantekidis et al., "Deep Learning for Financial Time Series Forecasting."

[3] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, 1997.

[4] A. Borovykh et al., "Conditional Time Series Forecasting with CNNs."

[5] yfinance Python library, https://pypi.org/project/yfinance/

[6] PyTorch Documentation, https://pytorch.org/docs/stable/

Project Group Members:

Archith Sunil	TCR24CS015
Bhagath P. R.	TCR24CS019
David Chacko Binoy	TCR24CS022
Devanandan J. Y.	TCR24CS023
Harinarayanan R.	TCR24CS033
Joseph Mathew	TCR24CS038